Project Proposal: Real-Time IoT Data Pipeline

Team Leader: Rana Shaaban Abdelqader Mohamed

1. Project Overview

This project builds an end-to-end real-time IoT data pipeline. Simulated sensors generate temperature and humidity readings, which are pushed through a message queue (Apache Kafka), processed through both a **batch** path (ETL into a data warehouse) and a **streaming** path (real-time alerts), and finally visualized on a live dashboard.

The project introduces the team to orchestration, real-time analytics, and cloud-native data processing in a realistic Data Engineering workflow.

2. Project Analysis

2.1 Problem Statement and Needs Assessment

Problem Statement: IoT sensors produce continuous streams of readings, but raw readings are only useful if they are collected reliably, cleaned, stored for analysis, and monitored in real time. Without a pipeline, abnormal values (for example, a temperature spike) are noticed late, and the data cannot be analyzed later. The gap is between raw sensor output today and a monitored, analyzable data flow.

Needs Assessment:

Type	Need
Felt need (team and lecturer)	A working end-to-end pipeline that the team can build and demonstrate, with hands-on practice in Kafka, ETL, streaming, and dashboards
Felt need (end user)	Immediate alerts when temperature or humidity cross a threshold, and a live view of sensor trends
Normative need (course requirements)	Both a batch and a streaming pipeline, the weekly deliverables, and all documents uploaded to GitHub

2.2 Stakeholder Analysis

Stakeholder	Impact / Power	Core Interests and Expectations
Lecturer	High / High	A complete, working project that meets the requirements and weekly deliverables, with clear documentation
Project team (5 members)	High / High	A fair task split, learning real Data Engineering tools, and a successful final demo
Dashboard users (monitoring operators)	Medium / Low	Accurate, easy-to-read live metrics and timely alerts

2.3 Feasibility Study

- **Technical feasibility:** The required tools (Python, Flask, Kafka, Snowflake, Streamlit) are widely documented and can run on the team's own machines, and the data is simulated, so no hardware is needed.
- **Operational feasibility:** Each member owns one clear component with defined hand-offs, so the five parts can be built in parallel during the 4 weeks.
- **Economic feasibility:** The tools are open source or free, and Snowflake can run on a trial account, so the direct cost is close to zero. The main cost is the team's time over 4 weeks.

3. Project Design

3.1 Objectives

SMART objective: Within 4 weeks, build and demonstrate a pipeline that generates temperature and humidity readings every 5 seconds, loads cleaned data into Snowflake with at least 99% completeness, raises an alert within 5 seconds of a threshold breach, and shows sensor trends, anomaly counts, and alert history on a live dashboard.

Supporting objectives:

- Simulate IoT sensor data (temperature and humidity) every 5 seconds with a defined schema.

- Ingest the data through a Flask API and a Kafka producer into Kafka topics.
- Build a batch ETL pipeline that cleans, transforms (flag anomalies, compute averages), and loads data into a data warehouse (Snowflake).
- Build a streaming pipeline that applies threshold rules and raises alerts in real time.
- Build a live dashboard showing sensor trends, anomaly counts, and alert history.
- Document the system and present findings and performance in a final report.

3.2 Scope

In scope

- Simulated sensors only (no physical hardware).
- Two data fields: temperature and humidity, plus sensor ID, timestamp, and location.
- Batch ETL path and streaming alert path, both fed from Kafka.
- One dashboard covering processed and streamed data.
- Documentation, final report, and final presentation.

Out of scope

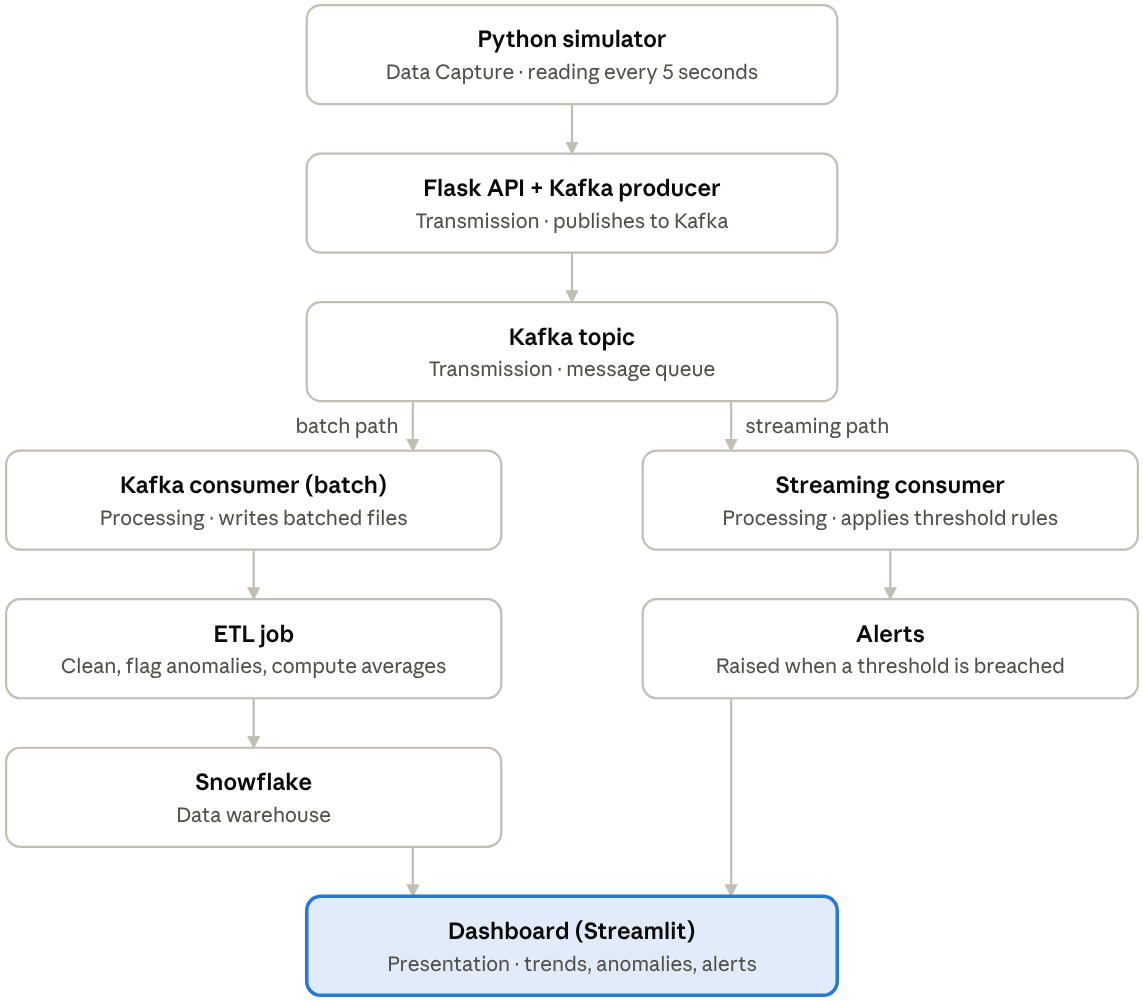
- Real IoT hardware and device firmware.
- Machine learning or predictive models.
- Production-grade security, scaling, and cost optimization.

3.3 Technical and System Architecture

The system has four layers. Data moves from the simulator, through Kafka, into two parallel paths (batch and streaming), and ends on the dashboard.

- **Data Capture Layer:** A Python simulator generates temperature and humidity readings every 5 seconds, following the schema (sensor ID, timestamp, temperature, humidity, location).
- **Transmission Layer:** A Flask API receives the readings and a Kafka producer publishes them to a Kafka topic.
- **Processing Layer, batch path:** A Kafka consumer writes batched files, and the ETL job cleans, flags anomalies, computes averages, and loads the result into Snowflake.
- **Processing Layer, streaming path:** A separate Kafka consumer applies threshold rules in real time and raises alerts.
- **Presentation Layer:** A Streamlit dashboard reads processed data from Snowflake and the alert stream to show trends, anomaly counts, and alert history.

One Kafka topic feeds both the batch and streaming paths



system architecture · 4 layers, 2 paths

Readings enter through the Flask API and Kafka, then split into a batch path that ends in Snowflake and a streaming path that raises alerts. The dashboard reads from both.

3.4 Technologies

Layer	Technology
Data simulation and scripting	Python, Pandas
Ingestion API	Flask
Message queue	Apache Kafka
Batch ETL and storage	Python ETL, Snowflake (data warehouse)

Layer	Technology
Real-time processing	Kafka consumer with threshold rules (Azure Stream Analytics as an alternative)
Dashboard	Streamlit (Power BI or Grafana as alternatives)
Version control and delivery	Git and GitHub

3.5 Deliverables

Week	Focus	Main Deliverables
1	Data simulation and ingestion	Python generator script, sample data logs, data schema documentation, environment setup notes
2	Batch data pipeline (ETL)	ETL script, processed dataset in storage, data validation report, ETL documentation
3	Streaming pipeline with alerts	Streaming pipeline setup, alert logic code and output, real-time query configurations, streaming architecture documentation
4	Dashboard and final report	Dashboard screenshot or live demo, final project report, presentation slides, system architecture summary

3.6 Timeline and Risk Matrix

Phased roadmap (4 weeks): Week 1: data simulation and ingestion | Week 2: batch ETL into Snowflake | Week 3: streaming pipeline and alerts | Week 4: dashboard, final report, and presentation. The detailed schedule, milestones, and Gantt chart are in the Project Plan document.

Top risks:

Risk	Likelihood	Impact	Designed Mitigation
Team is new to Kafka, Snowflake, and streaming	High	Medium	Share tutorials in Week 1 and pair members on first tasks

Risk	Likelihood	Impact	Designed Mitigation
Kafka setup problems	Medium	High	Use a documented Docker setup; fall back to file-based ingestion if needed
Schema changes after components are built	Medium	High	Agree and freeze the schema in Week 1; changes need team leader approval
Delays from dependencies between members	High	Medium	Agree interfaces early and use sample files to work in parallel

The full risk register is in the Risk Assessment and Mitigation Plan document.

4. Team

#	Member	Role
1	Rana Shaaban Abdelqader Mohamed	Team Leader, Dashboard
2	Abdelrahman Sayed Taha Morsi	Kafka Consumer and Batched Files
3	Mostafa Ahmed Mahmoud Shaban	Data Simulator, Streaming and Alerts
4	Shahd Safwat Mohamed	ETL to Snowflake
5	Waleed Hamouda Abd Elfattah Mahmoud	Flask API and Kafka Producer

Full responsibilities are listed in the Task Assignment and Roles document.

5. Expected Outcome

A working pipeline that continuously generates sensor data, stores cleaned and enriched data in a warehouse, raises real-time alerts on threshold breaches, and displays everything on a live dashboard, supported by complete documentation stored on GitHub.